ANDROID DEVELOPMENT

SIRAJ DUJANA QURESHI

Contents

Introduction to Android Development 2

Introduction to Layouts 6

Android Widgets Overview 11

Understanding Intents in Android 14

Jetpack Compose 15

Thread(note not in syllabus): 19

Essential Attributes: 20

QUIZ Questions 1: 21

Mid Questions: 22

Introduction to Android Development

Short History of Android

- 2003 → Android Inc. founded.
- 2005 → Google bought Android.
- 2008 → First Android phone (HTC Dream).
- 2009–2018 → Versions named after desserts 🍩🍰🍦.
- 2019 onward → Simple numbers (Android 10, 11, 12…).
- Now → Biggest mobile OS in the world 🌍.

1. Understand the Android Ecosystem

Android Ecosystem = everything needed to make Android apps.

It includes:

- Android OS (running on phones, TVs, watches).
- Android Studio (the main tool for developers).
- Java/Kotlin (programming languages used).
- Libraries & SDKs (ready-made code and tools).
- Google Play Store (where apps are published).
- Think of it as a "world" where developers, tools, and devices work together.

2. Install and Configure Android Studio

Android Studio is the official IDE (like a workshop where you build apps).

Steps:

- Download from → developer.android.com/studio
- Install Java (JDK) if required.
- Open Android Studio → it will install SDKs (software kit to build apps).
- Create a Virtual Device (Emulator) or connect your phone with USB.
- After setup, you're ready to write code and run apps.

Explore Android Architecture and Runtime

Architecture:

Architecture means the overall structure and design of a system, how different parts are arranged and how they work together.

Example:

- House architecture → shows rooms, walls, doors, and how they connect.
- Computer/Android architecture → shows layers (apps, libraries, hardware) and how they interact.

ndroid architecture is divided into five main layers, each with a specific role:

Android Architecture

1. Linux Kernel (Hardware Abstraction)

- Acts as a bridge between hardware and software.
- Manages device drivers (camera, Wi-Fi, Bluetooth, display, etc.), memory, and power.
- Provides security (process isolation, permissions).
- Example: When your app uses the camera, the request goes through the kernel to talk to the actual camera hardware.

2. Libraries

Collection of pre-written code in C/C++ used by Android.

Common libraries:

- SQLite → Manages app databases (e.g., WhatsApp stores chat history).
- WebKit → Renders web pages inside apps (e.g., in-app browser).
- OpenGL → For 2D/3D graphics (e.g., games).
- Example: If your app saves user login info, it may use SQLite as the database.

3. Android Runtime (ART)

Executes your app code.

Replaced the older Dalvik VM.

Features:

- Ahead-of-Time (AOT) & Just-in-Time (JIT) compilation for faster performance.
- Garbage Collection to manage memory automatically.
- Example: When you tap to open an app, ART converts the app's code (.dex files) into machine language so the CPU can run it.

4. Application Framework

Provides high-level APIs that developers use.

Important managers:

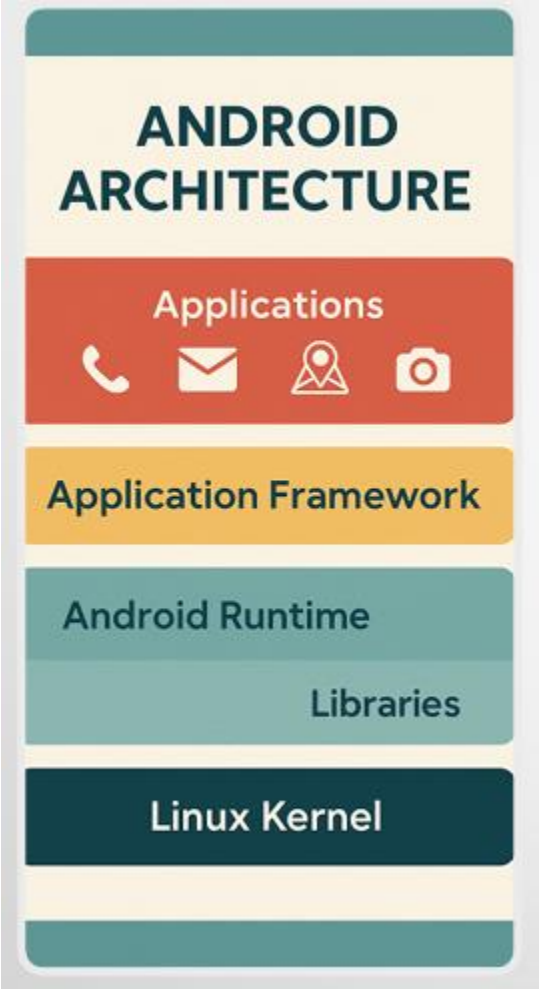
- Activity Manager → Handles app screens (Activities).
- Content Providers → Share data between apps (e.g., Contacts app data).
- Notification Manager → Manages push notifications.
- Location Manager → Provides GPS data.
- Example: The Activity Manager ensures that when you switch from WhatsApp to YouTube, one activity pauses while another resumes.

5. Applications

The top layer of Android architecture.

Includes:

- Built-in apps (Phone, Contacts, Messages, Settings).
- User-installed apps (Facebook, Instagram, your HelloWorld app).
- Example: When you install a calculator app, it becomes part of this layer and interacts with the lower layers via framework APIs.



Running Your First Android App

To run your first app in Android Studio, follow these steps:

1. Use Emulator or Physical Device

- Emulator → A virtual Android phone inside your computer (created in Android Studio).
- Physical Device → Your own Android phone (enable Developer Options → USB Debugging to connect).

Example: If you don't have an Android phone handy, you can test your HelloWorld app on the Emulator.

2. Build and Run from Android Studio

- Click the Run ▶ button in Android Studio.
- Select your Emulator or connected device.
- Android Studio compiles the project and installs the app automatically.

Example: When you press Run, the HelloWorld app will appear on the emulator screen.

3. Check for Errors in Logcat

- Logcat = real-time system log viewer in Android Studio.
- Shows errors, warnings, and messages from your app.
- Useful for debugging (finding why the app crashed).

Example: If your app stops, Logcat may show: "NullPointerException at MainActivity", so you know where to fix the code.

Introduction to Layouts

Layouts in Android define how UI elements (buttons, text, images, etc.) are arranged on the screen.

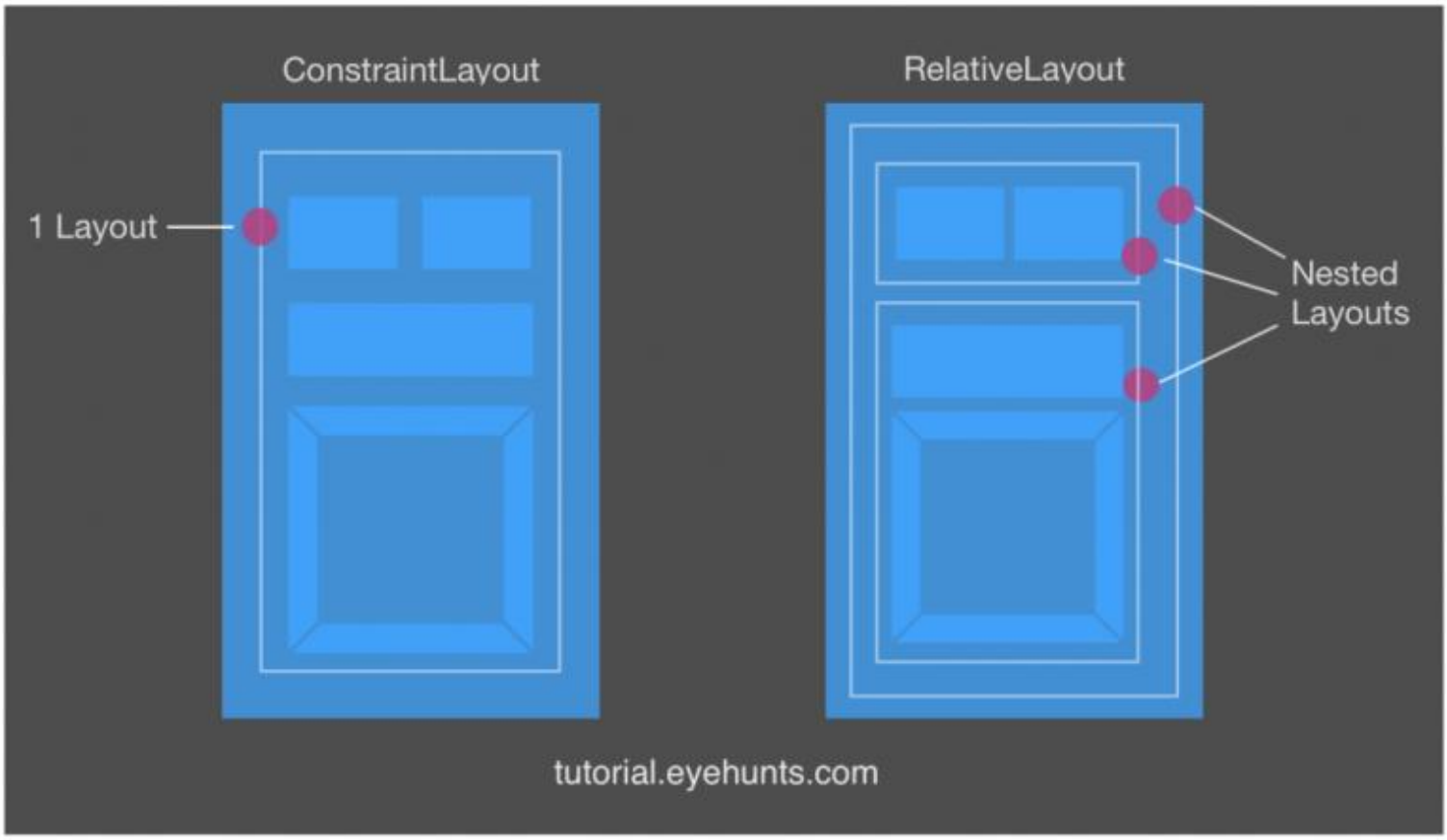
They are written in XML files inside the res/layout folder.

1. **ConstraintLayout**

ConstraintLayout has flat view hierarchy unlike other layouts, so does a better performance than relative layout. Yes, this is the biggest advantage of Constraint Layout, the only single layout can handle your UI. Where in the Relative layout you needed multiple nested layouts (LinearLayout + RelativeLayout).

- Flexible and powerful layout.
- Lets you position UI elements by creating constraints (e.g., align button to parent, to another view, or center).
- Reduces nested layouts → better performance.
- Example: A button centered on the screen:

```
<androidx.constraintlayout.widget.ConstraintLayout

   xmlns:android="http://schemas.android.com/apk/res/android"

   xmlns:app="http://schemas.android.com/apk/res-auto"

   android:layout_width="match_parent"

   android:layout_height="match_parent">


   <Button

      android:text="Click Me"

      android:layout_width="wrap_content"

      android:layout_height="wrap_content"

      app:layout_constraintTop_toTopOf="parent"

      app:layout_constraintBottom_toBottomOf="parent"

      app:layout_constraintLeft_toLeftOf="parent"

      app:layout_constraintRight_toRightOf="parent"/>

</androidx.constraintlayout.widget.ConstraintLayout>
```



2. RelativeLayout

- Places views relative to each other or to the parent.
- You can align views above, below, left, right, or center.
- Now mostly replaced by ConstraintLayout.
- Example: TextView above a Button:

```
<RelativeLayout
  xmlns:android="http://schemas.android.com/apk/res/android"
  android:layout_width="match_parent"
  android:layout_height="match_parent">

  <TextView
    android:id="@+id/title"
    android:text="Welcome"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_centerHorizontal="true"/>
```

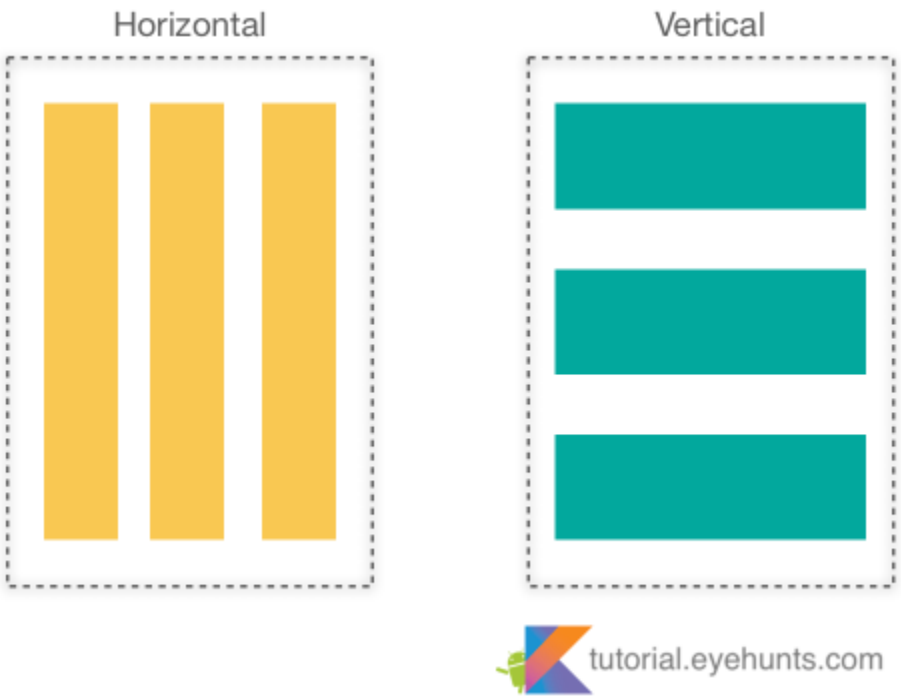
```
    <Button
        android:text="Start"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_below="@id/title"
        android:layout_centerHorizontal="true"/>
</RelativeLayout>
```

3. LinearLayout

- Arranges views in a single direction:
- Vertical (top → bottom)
- Horizontal (left → right)
- Simple and commonly used.
- Example: Vertical list of buttons:

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
```

```
    <Button android:text="One"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>
    <Button android:text="Two"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>
</LinearLayout>
```



4. GridView

- Displays items in a scrollable grid (rows + columns).
- Commonly used for image galleries.
- Needs an Adapter to supply data (text/images).
- Example: Grid with 3 columns:

```
<GridView
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/gridView"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:numColumns="3"
```

```
    android:verticalSpacing="10dp"
    android:horizontalSpacing="10dp"/>
```

5. TableLayout

- Organizes UI into rows and columns, like an HTML table.
- Each row = <TableRow>.
- Good for forms and structured data.
- Example: Simple login form:

```
<TableLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content">

    <TableRow>
        <TextView android:text="Username:"/>
        <EditText android:hint="Enter username"/>
    </TableRow>

    <TableRow>
        <TextView android:text="Password:"/>
        <EditText android:hint="Enter password"
            android:inputType="textPassword"/>
    </TableRow>
</TableLayout>
```

Android Widgets Overview

Widgets in Android are the interactive components used to build the user interface of applications.

They allow user interaction, such as displaying information, taking input, or triggering actions.

1. Understanding Android Widgets

Core Widgets → Basic UI components:

TextView → Displays text.

EditText → Captures user input.

Button → Triggers actions.

ImageView → Displays images.

Customization → Widgets can be styled with attributes like:

Layout size (width/height)

Padding and margins

Text color, size, and style

Interactive Functionality → Widgets handle clicks, inputs, and other user events.

Custom & Compound Widgets → Developers can extend or combine widgets for advanced features (e.g., a custom login box combining EditText + Button).

2. TextView – Displaying Text

Basics:

A non-editable widget for showing static text.

Customization Options:

Text size, color, font style, gravity (alignment), and background.

Dynamic Updates:

Text can be updated in code using setText().

Advanced Features:

Multiline text

HTML formatting

Clickable links

Drawable icons (e.g., left icon in a label)

Example (XML):

```
<TextView
   android:layout_width="wrap_content"
   android:layout_height="wrap_content"
   android:text="Hello Android!"
   android:textSize="20sp"
   android:textColor="#000000"
   android:padding="10dp"/>
```

3. EditText – Capturing User Input

Versatile Input Types:

Supports text, numbers, passwords, email, phone, etc.

User Guidance & Customization:

Attributes like hint, maxLines, textSize, and textColor.

Data Retrieval & Validation:

Input can be retrieved in code using:

```
String input = editText.getText().toString();
```

Validate input and show error messages for accuracy.

Integration in Forms:

Used with Buttons/TextViews to create login forms, feedback forms, etc.

Example (XML):

```
<EditText
   android:layout_width="match_parent"
   android:layout_height="wrap_content"
   android:hint="Enter your name"
   android:inputType="textPersonName"/>
```

4. Button – Triggering Actions

Basics:

Executes an action when clicked.

Commonly used for navigation and form submission.

Customization:

Buttons can be styled with text, colors, icons, and backgrounds.

Handling Events:

In XML → android:onClick="buttonClicked"

In Java/Kotlin →

```
button.setOnClickListener(v -> {
    // Action here
});
```

Button States:

Supports pressed, focused, and disabled states.

Styled using selectors for different states.

Example (XML):

```
<Button
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Submit"
    android:background="@drawable/button_selector"/>
```

Understanding Intents in Android

Intent = Message/Request that tells Android what you want to do.

It helps different parts of an app (activities, services, broadcast receivers) or even different apps talk to each other.

Real Examples:

- In Google Maps, when you tap on a restaurant's website → it opens your browser (Chrome, Firefox, etc).
- In FoodPanda, when you tap "Call Restaurant" → it opens the Phone Dialer.
- In Instagram, when you tap "Share to…" → it gives you options like WhatsApp, Gmail, SMS, Messenger to share.
- In Uber, when you see a location and tap "View on Map" → it opens Google Maps or any maps app installed.
- In Camera apps, when you click "Gallery" → it opens your phone's Photos app.

Types of Intents

- Explicit Intent → You tell exactly which component (Activity) to open.
    - Explicit Intent = Go from Login → Dashboard (inside your own app).
- Implicit Intent → You ask Android to find someone who can handle your request (system or other apps).
    - Implicit Intent = Open Maps, Browser, or Share something (other apps can handle it).

Why Intents Are Important

- Explicit intents → Move between screens in your app.
- Implicit intents → Use phone's features (browser, maps, camera) or share data with other apps.

Jetpack Compose

- **Jetpack Compose** = A modern toolkit to build Android UI faster and easier.
- Modern UI toolkit by Google for building Android apps with Kotlin.

Instead of writing XML layouts + Java/Kotlin UI code separately, you can directly build UIs using Composables (functions that create UI elements).

Think of Composable functions as LEGO blocks — you combine them to make your app screen.

What is Jetpack Compose?

- Toolkit for native Android UI.
- Uses reactive programming → UI updates automatically when data changes.
- Composable functions = reusable UI blocks (Text, Button, Image).

Benefits

- Declarative → Build UI by calling functions.
- Faster Development → XML + Kotlin combined in one place.
- Compatible → Works with existing Android views.
- Idiomatic Kotlin → Uses Kotlin's full power.
- Easy Maintenance → Code is cleaner and reusable.

Challenges Solved

- Coupling Issues → Less dependency between UI & logic.
- Dynamic UI → Easier to update without XML headaches.
- Single Language → Kotlin only (no XML + Java mix).
- Lifecycle Management → UI reacts to app lifecycle automatically.

Basic Functions

- @Composable → Defines a UI element.
- Column / Row / Box → Layouts (vertical, horizontal, overlapping).
- Spacer / Padding → Control spacing.
- LazyColumn → Large, scrollable lists.
- @Preview → See UI in IDE without running app.

Why It's Revolutionary

- Simplifies development (fewer files, less boilerplate).
- Readable & Maintainable code.
- Community growth → More support, libraries, and best practices.

Basic UI Components

1. Text

- Used to show any text (headings, labels, descriptions).

- Supports styles (bold, italic), colors, and sizes.

Real Example:

- WhatsApp → chat messages.
- Instagram → post captions.
- YouTube → video title & views.

2. Button

- A clickable component that triggers an action.
- Can have text, icons, or both.

Real Example:

- Login button in Facebook app.
- "Send" button in Gmail.
- "Like" button in Instagram (if designed as Compose).

Layout Components (Arranging UI Elements)

1. Column

- Arranges items vertically (top to bottom).

Real Example:

- A settings screen (each option is stacked).
- WhatsApp chat list (name + last message + time).

2. Row

- Arranges items horizontally (side by side).

Real Example:

- Navigation bar in YouTube (Home | Shorts | Subscriptions).
- Instagram story thumbnails (profile pics in a row).

3. Box

- Lets you place items on top of each other (overlapping).

Real Example:

- Instagram profile pic with "online dot" overlay.
- Background image with text written on top.

Advanced UI Components

1**. LazyColumn**

- Special column for long, scrollable lists (loads only visible items = saves memory).

Real Example:

- Instagram feed scrolling.
- YouTube video list.
- Shopping app product lists.

2. Card

- A container with shadow & rounded corners.
- Used for grouping content.

Real Example:

- Google News article preview.
- FoodPanda restaurant menu cards.
- Banking apps showing "Balance Card".

3. Scaffold

- Provides a basic app structure: top bar, bottom bar, floating button, body.

Real Example:

- Gmail app (Top app bar = “Search mail”, FAB = Compose button, Bottom bar = navigation).

Animation Components

1. AnimatedVisibility

- Show/hide elements smoothly.

Real Example:

- Floating button disappears when you scroll (Gmail compose button).

2. Transition

- Animate changes between states.

Real Example:

- Instagram heart animation when you double tap a post.

3. Decay Animation

- Natural fling/scrolling effect.

Real Example:

- Twitter feed slows down naturally when you scroll fast.
-

Accessibility Features

- ContentDescription → Describes images for screen readers.
    - Example: Profile pic reads "Picture of Faisal".
- Modifier Semantics → Adds extra info for accessibility.
    - Example: Button labeled "Send Message".
- Accessible TextFields → Proper labels for inputs.
    - Example: Login field says "Enter your email".

Why Jetpack Compose Rocks

- Faster UI building (no XML hassle).
- More readable and maintainable code.
- Follows Material Design (modern look).
- Already used in real apps (Twitter, Airbnb, Play Store updates).

Jetpack Compose – Streamlining Android UI

- Robust Composables → Ready-made UI blocks (Text, Button, Card, LazyColumn).
- Intuitive Modern UIs → Follows Material Design, looks clean and user-friendly.
- Ease & Flexibility → Fast to build, easy to customize, supports innovation.

Thread(note not in syllabus):

In computer science, a thread is the smallest unit of execution within a process.

Think of it like this:

A process is like an application (e.g., your browser).

A thread is like a worker inside that application, performing tasks.

Key Points:

Definition:

A thread is a lightweight process that shares the same memory and resources of its parent process but can run independently.

Purpose:

Threads are used to perform multiple tasks at the same time (concurrency).

Example: In a web browser:

One thread loads the webpage.

Another thread plays a video.

Another thread handles user input.

Types of Threads:

User-level threads → Managed by the programmer using libraries.

Kernel-level threads → Managed directly by the operating system.

Benefits:

Faster task execution (parallelism).

Better CPU utilization.

Allows multitasking within a single program.

Essential Attributes:

- id → Unique identifier for the view (e.g., @+id/button1).
- layout_width → Width of the view (e.g., match_parent, wrap_content).
- layout_height → Height of the view (e.g., match_parent, wrap_content).
- layout_margin → Space outside the view (top, bottom, left, right).
- padding → Space inside the view, between border and content.
- gravity → Positioning of content inside the view (like center, left, right).
- text / src → Text for TextView, or image source for ImageView.

The most essential ones are always:

- id
- layout_width
- layout_height

QUIZ Questions 1:

- A list down the components/layers of android architecture
  - Linux Kernel → Hardware abstraction, drivers, security, memory & process management.
  - Libraries → Pre-built C/C++ libraries like SQLite (database), WebKit (browser engine), etc.
  - Android Runtime (ART) → Executes app code and manages memory.
  - Application Framework → Provides APIs for developers (Activity Manager, Window Manager, Content Providers).
  - Applications → Both built-in (Dialer, SMS) and user-installed apps.
- Which method is used to set root view of an activity in android?
  - setContentView()
- List down the essential attributes of any widget/view
  - id
  - layout_width
  - layout_height
- Why we need layout weight and weight sum in the linear layout
  - To divide space among child views proportionally.
- Why orientation attribute is necessary in linear layout
  - To decide arrangement of views (vertical or horizontal).
- Why we need find view by id function in android
  - To access a UI element in code by its ID.
- What is the main difference between linear layout and relative layout in the therms of useability
  - LinearLayout → Simple, arranges views in one direction (row/column).
  - RelativeLayout → Flexible, places views relative to each other or parent.
- Why we prefer constraint layout on linear layout
  - Because it is more flexible, handles complex UI with fewer nesting → better performance.
- What is jetpack compose and why we need it
  - Jetpack Compose = Modern UI toolkit using Kotlin.
  - Needed because it makes UI building faster, easier, and without XML.

Mid Questions:

- Layered architecture
- Intent theory + example code

Coding portion

- login screen using linear layout

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <!-- Username field -->
    <EditText
        android:id="@+id/etUsername"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Username" />

    <!-- Password field -->
    <EditText
        android:id="@+id/etPassword"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Password"
        android:inputType="textPassword" />

    <!-- Login Button -->
    <Button
        android:id="@+id/btnLogin"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Login" />

</LinearLayout>
```

- Button functionality(validation) or intent (from one screen to anther screen) only java

```
LoginActivity.java
package com.example.loginapp;

import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

public class LoginActivity extends AppCompatActivity {

```

```
•      EditText etUsername, etPassword;
•      Button btnLogin;
•
•      @Override
•      protected void onCreate(Bundle savedInstanceState) {
•          super.onCreate(savedInstanceState);
•          setContentView(R.layout.activity_login); // your XML with LinearLayout
•
•          etUsername = findViewById(R.id.etUsername);
•          etPassword = findViewById(R.id.etPassword);
•          btnLogin = findViewById(R.id.btnLogin);
•
•          btnLogin.setOnClickListener(new View.OnClickListener() {
•              @Override
•              public void onClick(View v) {
•                  String user = etUsername.getText().toString();
•                  String pass = etPassword.getText().toString();
•
•                  // Simple validation
•                  if (user.isEmpty() || pass.isEmpty()) {
•                      Toast.makeText(LoginActivity.this, "Please enter all fields",
   Toast.LENGTH_SHORT).show();
•                  } else {
•                      // Validation success → move to HomeActivity
•                      Intent intent = new Intent(LoginActivity.this, HomeActivity.class);
•                      intent.putExtra("USERNAME", user); // pass username
•                      startActivity(intent);
•                  }
•              }
•          });
•      }
•  }
```

```
HomeActivity.java
package com.example.loginapp;

import android.os.Bundle;
import android.widget.TextView;

import androidx.appcompat.app.AppCompatActivity;

public class HomeActivity extends AppCompatActivity {

    TextView tvWelcome;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_home);

        tvWelcome = findViewById(R.id.tvWelcome);
```

```
        // Get username from intent
        String username = getIntent().getStringExtra("USERNAME");
        tvWelcome.setText("Welcome, " + username + "!");
    }
}
```

Summary:

- Toast is used for validation message.
- Intent is used to switch from LoginActivity → HomeActivity.
- putExtra is used to pass data (username).